

High-Performance Order Matching Engine — Project Overview

35,000+ lines of C++20 / 56 headers / 8 source files / 51 test files / 181 CTest targets

A C++20 low-latency matching engine drawing on institutional exchange design principles, built for sub-150ns processing latency and horizontal scalability.

1. Executive Summary

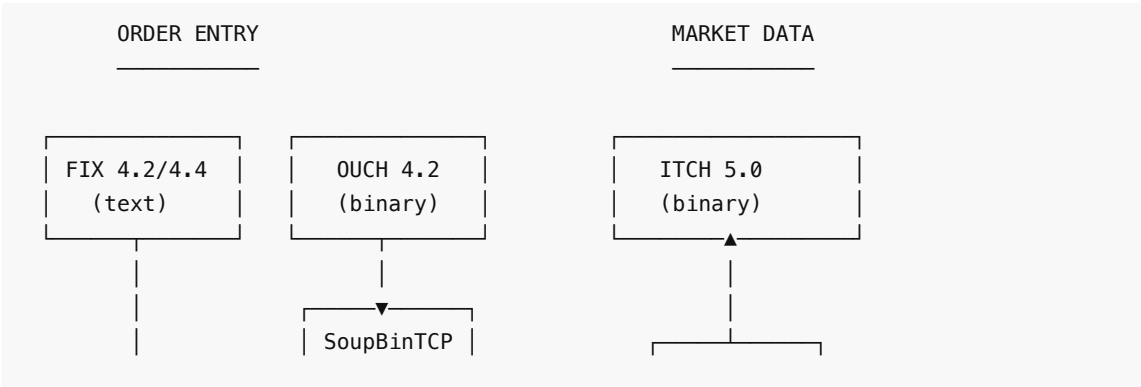
This is a C++20 low-latency order matching engine with institutional-grade architecture drawing on exchange design principles. It implements O(1) price-level lookup via `FlatPriceMap`, lock-free MPSC queues, thread-per-symbol horizontal scaling, CRC-32 journaling with deterministic replay, four wire protocols (FIX 4.2/4.4, OUCH 4.2, ITCH 5.0, SBE) over both real TCP and UDP transports, MoldUDP64 multicast with gap-recovery retransmission service, TLA+-verified safety invariants, cross-host log replication, and a complete operational stack including config management, webhook alerting, Prometheus metrics, and Docker deployment.

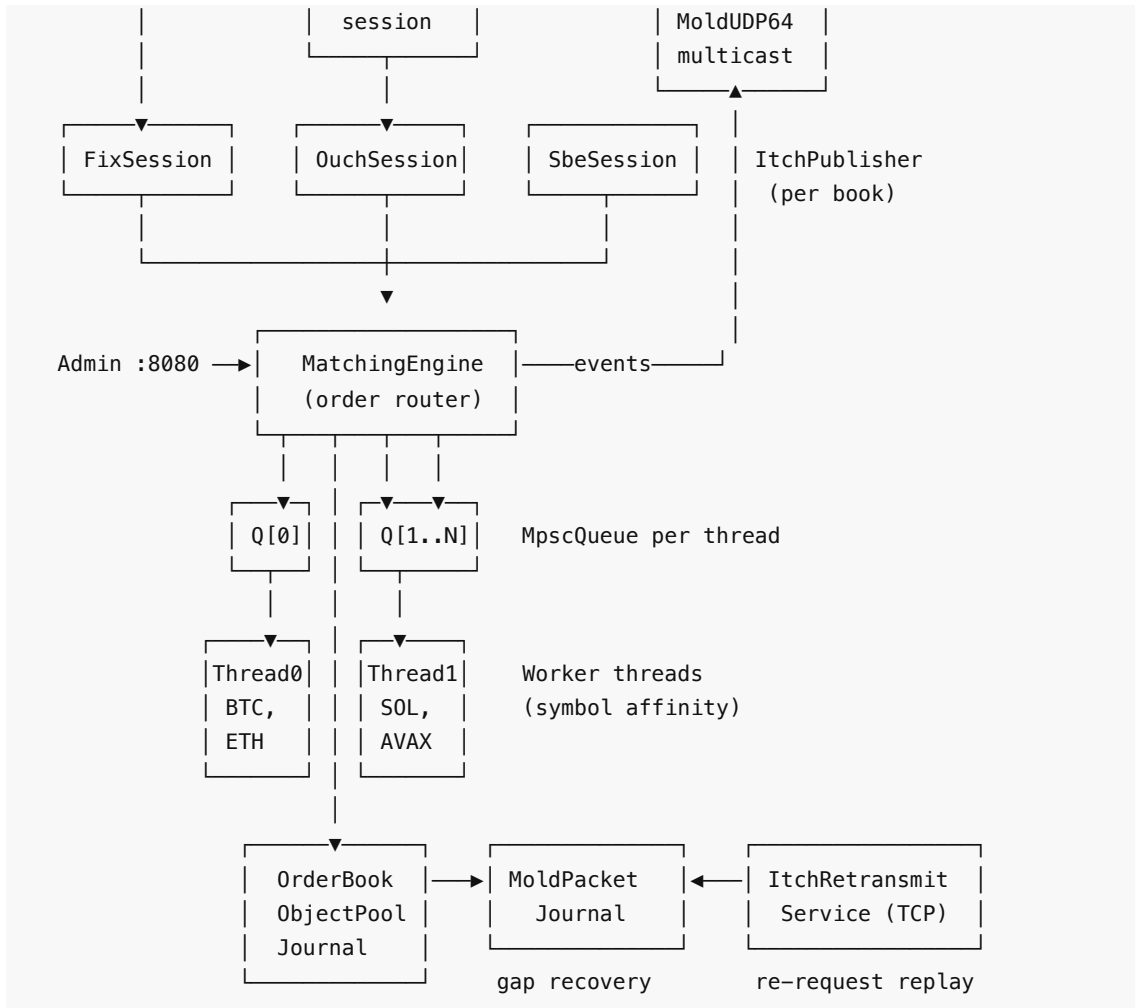
Codebase Statistics

Metric	Count
Total C++ LOC	~35,800
Header files (<code>include/</code>)	56
Source files (<code>src/</code>)	8
Test files	51
Individual test cases	181 CTest targets
TLA+ specifications	7 (454M+ states verified)
Documentation files	6 (plus architecture/benchmark docs)

2. Core Engine Architecture

System Topology





Order Types Supported

- **Standard:** Limit, Market
- **Time-In-Force:** IOC (Immediate-or-Cancel), FOK (Fill-or-Kill), DAY, GTD (Good-Til-Date)
- **Conditional:** Stop, StopLimit, TrailingStop
- **Institutional:** Pegged, Iceberg (hidden quantity), Hidden, PostOnly
- **Match Algorithms:** Price-Time FIFO and Pro-Rata allocation

Core Data Structures

Component	Purpose	Complexity
FlatPriceMap	Price-level lookup by tick index	O(1) insert/lookup
FlatHashMap	Robin-Hood open-addressing hash map	O(1) amortized
IntrusiveList	Doubly-linked order queue per price level	O(1) insert/remove
ObjectPool	Pre-allocated slab allocator for order nodes	O(1) alloc/dealloc
RingBuffer	Cache-line aligned lock-free ring buffer	O(1) push/pop
MpscQueue	Multi-producer, single-consumer lock-free queue	O(1) push/pop

3. Concurrency & Networking

Thread-Per-Symbol Partitioning

N worker threads with independent lock-free `MpscQueue` ring buffers. Orders are deterministically routed by `hash(symbolId) % numThreads`, eliminating cross-thread contention on the hot path.

Multi-Protocol Order Entry

All four protocols dispatch into the same `MatchingEngine`.

- **FIX 4.2 / 4.4**: ASCII text + checksum, full session-layer state machine. `TransactTime` validation and version negotiation per accepted `BeginString`.
- **OUCH 4.2**: NASDAQ binary order entry, big-endian fixed-width.
- **SBE (Simple Binary Encoding)**: FIX-TG schema-driven binary (CME / ICE style). Forward and backward compatibility. Encode speed of ~1 ns/op.
- **SoupBinTCP**: TCP envelope wrapping OUCH on the wire.

Market Data Stack

- **ITCH 5.0 Publisher**: Converts engine events to ITCH 5.0 frames.
- **MoldUDP64 Multicast**: Batched publisher with MTU auto-flush + gap-detecting subscriber.
- **Gap Recovery**: Bounded ring of journaled MoldUDP64 messages backing a SoupBinTCP-over-TCP retransmission service.
- **Shared Memory IPC**: L2 market data distribution to co-located consumers via `shm_open`.

4. Regulatory & Risk Management

Feature	Implementation
Self-Match Prevention (SMP)	Cancel-Taker strategy — prevents wash trading
Circuit Breakers	Configurable % price band halts (volatility protection)
OTR Monitoring	Real-time Order-to-Trade ratio tracking per participant
Kill Switch	Instant cancellation of all orders per participant across all symbols
Pre-Trade Risk Limits	Max order size, notional value, and position limits per participant
Rate Limiting	Token-bucket throttling at ingress (configurable rate + burst)
Queue Backpressure	Rejects orders when queue exceeds configurable threshold (default 80%)

5. Reliability & High Availability

CRC-32 Journaling

- Write-ahead log with atomic CRC-32 integrity on every entry.
- Crash recovery: replay stops at first corrupted/truncated record.
- Atomic checkpoint: snapshots active book state, rewrites journal.

- Deterministic replay via virtual clock (`setExpiryClock(ClockFn)`).

Cross-Host Log Replication

- `ReplicationCoordinator` — TCP log-shipping from primary to backup.
- `HeartbeatMonitor` — configurable failure detection timeout.
- `LeaderLease` — epoch-based fencing prevents split-brain.
- Automatic backup promotion on primary failure detection.

6. Observability & Operations

Observability Features

- **Structured Logging:** Pluggable `StructuredSink` API (Null, JSON Stderr, Capturing).
- **Prometheus Metrics:** `MetricsRegistry` with `/prometheus` text-exposition endpoint. Tracks order flow, queue depth, latencies, and journal health.
- **Webhook Alerting:** `AlertDispatcher` with targets for Slack, PagerDuty, or HTTP webhooks.

Admin HTTP Server (Port 8080)

Endpoint	Purpose
GET /health	K8s liveness probe
GET /metrics	Internal counters (JSON)
GET /prometheus	Prometheus text exposition
GET /book?symbolId=0	L2 order book snapshot
GET /otr?participantId=1	Order-to-trade ratio

7. Performance Benchmarks

Measured using `HonestBenchmark` — a single deterministic order flow (50K orders, seed=42) fed through three paths on Apple Silicon ARM64, Clang C++20 -O2.

Three-Path Latency (identical order flow)

Path	What's Included	P50	P99	P99.9	Throughput
Core matching	OrderBook + STP + WashTrade + LULD	125 ns	333 ns	458 ns	6.6M ops/s
Engine wrapper	+ sequence alloc, rate limiter	84 ns	292 ns	417 ns	7.1M ops/s
Full-stack journal	+ GroupCommit (batch=64, fdatsync)	1,040 ns	2.7 ms	4.0 ms	22K ops/s

Binary Codec Microbenchmark

Codec	Message	ns/op	M ops/s
-------	---------	-------	---------

OUCH 4.2	EnterOrder encode (49B)	112.0	8.9
ITCH 5.0	AddOrder encode (36B)	34.1	29.4
SBE	NewOrderV1 encode (32B)	1.0	1015
SBE	NewOrderV1 decode (32B)	0.5	2128

8. Verification & Testing

Test Suite — 51 Executables, 181 CTest Targets

The testing infrastructure includes Unit, Functional, Integration, Chaos, Property, Shadow, and Benchmark testing categories. Key mechanisms:

- **Shadow Mode:** Dual-book divergence detection, validating FIFO compliance.
- **Fault Injection:** 10+ injection points (short-writes, pool exhaustion, EAGAIN injection) with zero-cost overhead in production.
- **Coverage-Guided Fuzzing:** libFuzzer harness for protocol parsing and order flow.
- **Sanitizers:** ASan, UBSan, and TSan checks integrated into CI/CD.

Formal Verification (TLA+)

7 TLA+ specifications, model-checked with TLC:

- **MatchingEngine.tla** : 454M states verified. Zero violations for **NoNegativeQuantity** , **FIFO_Preservation** , and **GTD_Expiry_Correctness** .
- **Replication.tla** : Leader election & log shipping (preventing split-brain).
- **MpscQueue.tla** : Lock-free ring buffer linearizability.
- **SnapshotLocked.tla** : Mutex torn-snapshot prevention.

9. File Structure

include/	56 header files – core logic and networking
src/	8 source files – thin compilation units
tests/	42 test files – 181 CTest targets
benchmarks/	9 benchmark binaries
fuzz/	9 files – coverage-guided fuzzing harnesses
spec/	TLA+ formal specifications (7 specs)
tools/	CLI tools and data utilities
config/	Example configuration files
docs/	Runbooks, CapacityPlanning, ProductionReadiness

10. Remaining Work

The system is architecturally complete. The main remaining gaps require specialized hardware not typically available in standard environments:

- **x86 Bare Metal Benchmarks:** Needs multi-socket EC2 c5.metal instance.
- **Kernel Bypass Networking:** DPDK and io_uring seams are prepared, but require specific NICs (e.g., Solarflare/Onload) and tuning.

- **Replication.tla Verification:** Requires massive compute time for TLC model checker.
- **Wire-to-Wire Latency Validation:** Requires a multi-host test rig with hardware timestamping.

Developed for professional quantitative trading systems. C++20 · ~35,800 LOC · 51 test executables · 181 CTest targets · 454M TLA+ states verified